

项目名：外卖管理系统

需求与功能设计规格书

作者： 贾孟祺 丁雨晨 李艺涵 王烜铤

部门： 东大龙龙

时间： 2026/6/7

改版履历

版数	年月日	改版内容	承认	查阅	作成	备注
V1.0	2026/6/7	初版			贾孟祺	无
V1.1	2026/6/12	修改商家功能			贾孟祺	取消指派骑手功能

目录

1	前言	5
1.1	输入文档	5
1.2	相关文档	5
1.3	用语/术语定义	6
2	开发概要	7
2.1	开发的背景和目的	7
2.2	工作&运行环境	8
2.3	系统构成	10
2.4	功能整体图	11
2.5	用户分类和特征	12
3	外部接口需求	13
3.1	用户接口	13
3.2	硬件接口	13
3.3	软件接口	14
3.4	通讯接口	15
4	实现方式	16
4.1	多端统一认证功能	16
4.1.1	功能概要	16
4.1.2	注册/登录流程	17
4.1.3	相关功能	19
4.1.4	用户接口	20
4.2	顾客下单功能	20
4.2.1	功能概要	20
4.2.2	功能详细	20
4.2.3	相关功能	22
4.2.4	用户接口	22
4.3	商家管理功能	22
4.3.1	功能概要	22
4.3.2	功能详细	23
4.3.3	相关功能	23
4.3.4	用户接口	24
4.4	骑手配送功能	24
4.4.1	功能概要	24
4.4.2	功能详细	24
4.4.3	用户接口	25
4.5	WEBSOCKET 实时推送功能	25
4.5.1	功能概要	25
4.5.2	功能详细	26
4.5.3	相关功能	26
4.5.4	用户接口	26
4.6	数据库层功能	26
4.6.1	功能概要	26
4.6.2	功能详细	28
4.6.3	相关功能	28

4.6.4	用户接口	29
5	非功能需求	30
5.1	权限需求	30
5.2	兼容互换性	31
5.3	安全性	31
5.3.1	标准符合性:	31
5.3.2	安全设施需求:	32
5.3.3	安全保护措施与动作:	32
5.3.4	其他安全型需求:	34
5.4	健壮性	34
5.5	使用性（操作性）	35
5.6	效率性（性能）	36
5.7	维护性	39
5.8	移植性	40
5.9	用户文档	40
5.10	其他	40
6	使用和操作方法	41
6.1	环境设定	41
6.2	使用方法	41
6.2.1	安装依赖	41
6.2.2	启动服务	41
6.2.3	验证服务	41
6.2.4	退出系统	42
7	注意限制事项	43
7.1	制约 / 限制事项	43
7.2	假定事项	43
7.3	特记事项	43
8	附录	44
8.1	业务规则	44
8.2	术语表	44
8.3	数据流图	45
8.4	数据字典	46

1 前言

本文档为“全栈外卖管理系统（Takeaway Management System）”的需求与功能设计规格书，基于项目开发计划书和项目源代码实现进行功能模块分析与详细设计说明。

本系统是一套基于 B/S 架构的外卖管理平台，覆盖顾客下单、商家接单管理、骑手抢单配送的完整业务闭环，实现外卖业务流程的数字化、自动化和实时化。

1.1 输入文档

包括业务需求文档（外卖系统功能需求）、项目开发计划书（01-Plan.pdf）、技术栈约束（如 Python 3.10+/FastAPI/SQLAlchemy/Vue3/Element Plus/MySQL8+）以及《网络安全法》《个人信息保护法》等法规要求。

1.2 相关文档

项目开发计划书：全栈外卖管理系统开发计划（含技术栈、模块结构、数据库设计、业务流转逻辑等 5 大章节需求）

部署说明文档（README.md）：含环境要求、安装步骤、使用指南

requirements.txt + package.json：依赖包清单

1.3 用语/术语定义

术语	说明
Customer（顾客）	系统的消费者角色，可浏览菜品、管理收货地址、提交订单、查看历史订单
Merchant（商家）	系统的餐厅/店主角色，可管理菜品（增删改查/上下架）、处理订单（接单/拒单）
Courier（骑手）	系统的配送员角色，可查看可抢订单、主动抢单、确认配送完成
JWT	JSON Web Token，基于 JSON 的无状态身份认证令牌，HS256 算法签名
RBAC	Role-Based Access Control，基于角色的访问控制
SPA	Single Page Application，单页面应用，前端路由切换无整页刷新
ORM	Object-Relational Mapping，对象关系映射，将数据库表映射为 Python 对象
WebSocket	基于 TCP 的全双工实时通信协议，用于服务端向客户端实时推送消息
软删除	通过设置 `is_deleted=1` 标记逻辑删除，数据物理保留，不影响历史订单外键
事务 (Transaction)	数据库操作的原子性保证：多个操作要么全部成功，要么全部回滚（ROLLBACK）
状态机（State Machine）	订单状态的有序流转规则模型（0→1→2→3 / 0→4 / 0→5）
Bcrypt	密码单向哈希算法，将用户密码加密为不可逆的哈希值存储
CORS	Cross-Origin Resource Sharing，跨域资源共享，允许前端跨端口访问后端 API

2 开发概要

2.1 开发的目的和背景

开发目的：

本项目旨在构建一套面向餐饮外卖行业的全栈信息化管理平台，覆盖顾客下单、商家接单管理、骑手抢单配送的完整业务闭环，实现外卖业务流程的数字化、自动化和实时化。

开发背景：

随着移动互联网和电子商务的快速发展，外卖配送已成为居民日常消费的重要场景。据行业数据显示，中国外卖市场规模已突破万亿元，用户规模超过 5 亿。然而，大量中小型餐饮商家仍依赖人工电话接单、纸质记录、口头指派骑手等低效方式运营，存在以下痛点：

- 订单信息传递延迟，顾客等待时间不可预估
- 商家与骑手之间缺乏高效的协同机制
- 订单状态不透明，顾客无法实时了解配送进展
- 软删除与历史数据追溯困难

本系统从零开发一套完整的外卖管理系统，针对性地解决上述痛点，实现：

- 顾客在线浏览菜品并完成下单
- 商家实时接收订单提醒并处理订单
- 骑手主动抢单并完成配送确认
- 全流程订单状态实时同步与完整追溯

必要性与可行性：

- 经济可行性：系统采用开源技术栈（Python/FastAPI + Vue 3），开发成本低，后期维护成本可控
- 技术可行性：前后端分离架构成熟，WebSocket 实时通信技术已在工业界广泛验证
- 法规合规：系统遵循《网络安全法》《个人信息保护法》相关要求，密码加密存储，JWT 无状态认证保障用户数据安全
- 道德准则：公平对待顾客、商家、骑手三方利益，不设置算法歧视机制，骑手抢单机制公开透明

预期市场目标：

本系统定位于中小型餐饮商家及区域性外卖配送服务商，产品原型计划于 2026 年 6 月完成并进入内部测试阶段。

2.2 工作&运行环境

■ HW 环境

项目	最低配置	推荐配置
CPU	Intel Core i5 / AMD Ryzen 5	Intel Core i7 / AMD Ryzen 7
内存	8 GB DDR4	16 GB DDR4
磁盘	256 GB SSD	512 GB SSD
网络	100 Mbps LAN	1000 Mbps LAN

■ SW 环境

OS	最低版本	说明
Windows	10/11 (64-bit)	开发与部署均可
macOS	12+	开发与部署均可
Linux	Ubuntu 20.04+/CentOS 8+	推荐生产部署

Python3.10+与 Node.js 18+均为跨平台运行时，源码无需修改即可在上述 OS 运行

- 本系统为 B/S 架构 Web 应用，浏览器是用户端运行的必要条件。前端依赖现代浏览器 API（ES Module、WebSocket、CSS Grid/Flexbox）。

浏览器	最低版本	说明
Google Chrome	90+	首选，Vue Devtools 调试体验最佳
Microsoft Edge	90+ (Chromium 内核)	推荐，与 Chrome 同等兼容
Mozilla Firefox	90+	可用，部分 CSS 有细微差异
Apple Safari	15+	macOS / iOS 可用

不支持：Internet Explorer（所有版本）、国产浏览器「兼容模式」（IE 内核）、文本浏览器。

以下为系统部署运行**必须安装**的软件及软件包。缺失任何一项将导致系统无法正常启动或功能异常。

■ Server 运行环境：

类别	软件/包	最低版本	说明
语言运行时	Python	3.10+	后端代码执行环境
数据库	SQLite	3.x	开发环境，零配置
数据库	MySQL	8.0+	生产环境（开发环境无需安装）

■ Python 依赖包：

后端：

包名	版本	用途
fastapi	0.115.0	Web 框架
uvicorn	0.30.0	ASGI 服务器
sqlalchemy	2.0.35	ORM 框架
aiosqlite	0.20.0	SQLite 异步驱动
asyncmy	0.2.9	MySQL 异步驱动
python-jose	3.3.0	JWT 生成与验证
passlib	1.7.4	密码哈希 (bcrypt)
python-multipart	0.0.12	表单数据解析
pydantic	2.9.0	数据校验
pydantic-settings	2.5.0	环境配置管理

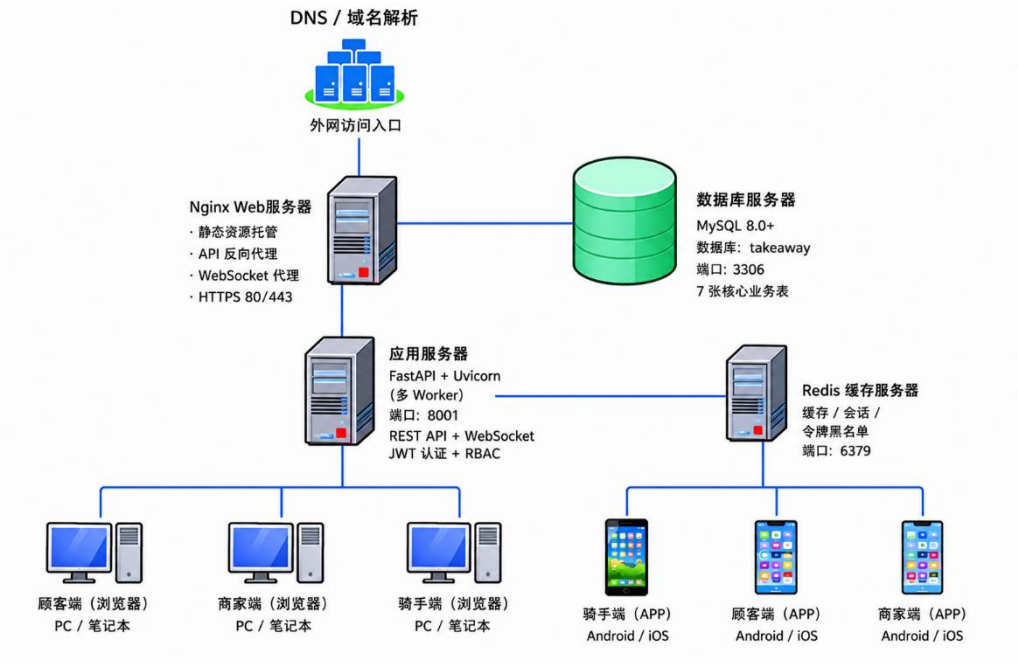
websockets	13.0	WebSocket 协议支持
------------	------	----------------

以上 11 个 Python 包均为产品运行依赖，Server 启动时必须全部已安装

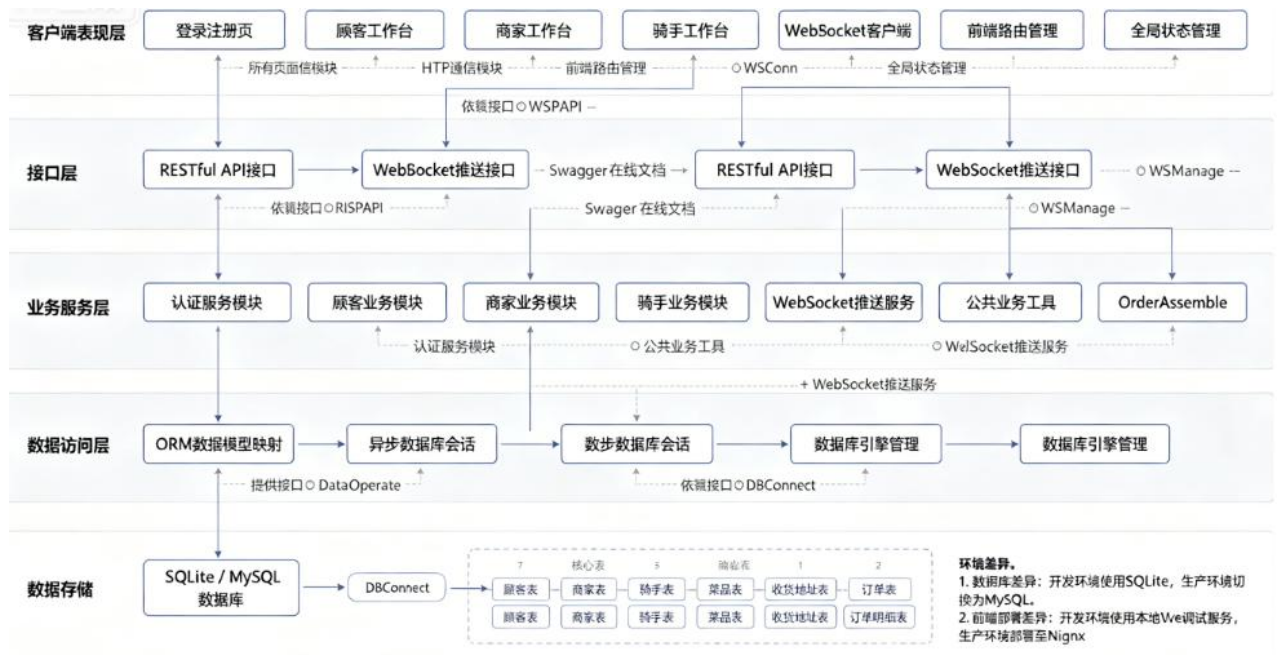
- 客户端运行环境（浏览器端）：
客户端为 Vue 3 SPA，所有前端框架及库经 Vite 构建打包为纯静态文件（HTML/CSS/JS），部署于 Web 服务器（Nginx / Apache / Vite Dev Server）。用户浏览器无需安装任何插件。

前端框架/库	版本	用途
Vue 3	3.4.0	前端框架
vue-router	4.3.0	SPA 路由
pinia	2.1.0	全局状态管理
element-plus	2.7.0	UI 组件库
@element-plus/icons-vue	2.3.0	图标集
axios	1.7.0	HTTP 请求库

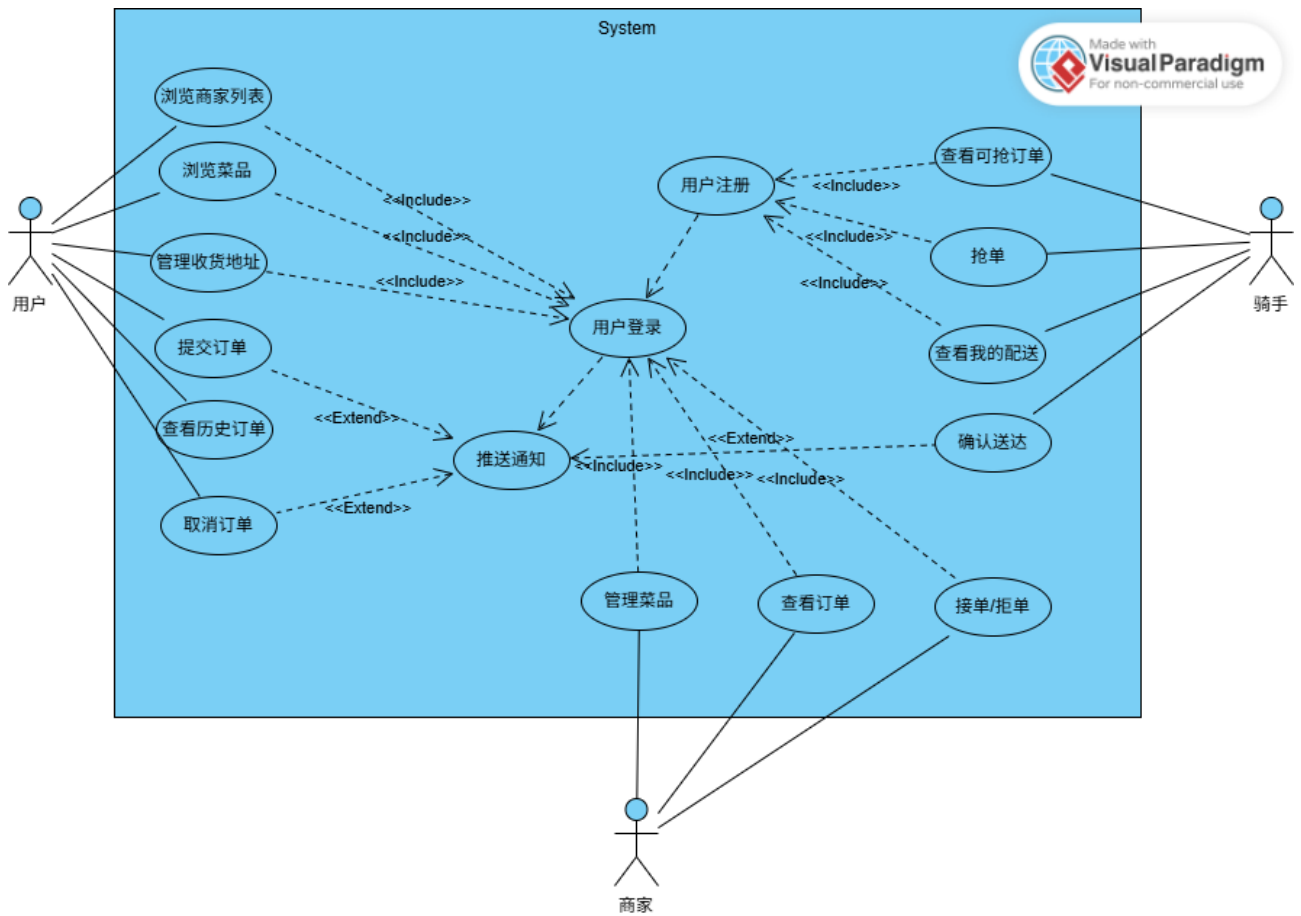
系统部署图：



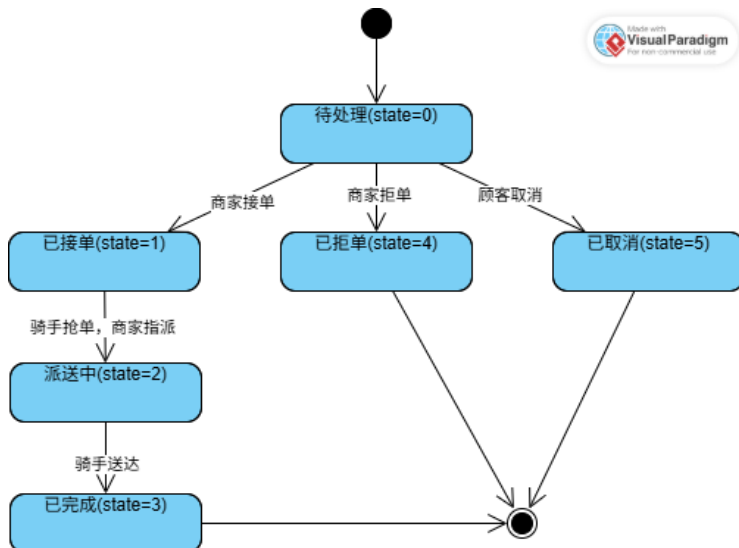
2.3 系统构成



2.4 功能整体图



补充事项：订单状态机如下



2.5 用户分类和特征

■ 用户一览表:

NO	用户	别名	描述
1	顾客(Customer)	消费者	浏览菜品、管理收货地址、购物车下单、查看历史订单、取消待处理订单
2	商家(Merchant)	店主	菜品管理(增删改查/上下架/软删除)、订单处理(接单/拒单)
3	骑手(Courier)	配送员	查看可抢订单列表、主动抢单、查看我的配送订单、确认送达完成配送

■ 各角色职责:

1. 顾客(Customer): 通过移动端页面浏览商家在售菜品(系统自动过滤已下架和已删除菜品)、管理个人收货地址(支持软删除,删除后不影响历史订单)、添加菜品至购物车并提交订单(系统采用原子事务保证数据一致性)、查看历史订单记录、取消待处理状态的订单。

2. 商家(Merchant): 通过 PC 工作台管理菜品全生命周期——新增、编辑(名称/价格/上下架状态)、软删除;实时查看订单列表并支持按状态筛选;处理订单:接单或拒单;通过 WebSocket 实时接收新订单推送通知。

3. 骑手(Courier): 通过移动端页面查看商家已接单但尚无骑手配送的“可抢订单”列表;主动发起抢单操作,同时骑手状态由空闲变为忙碌;查看已分配给自己的配送订单列表;配送完成后确认送达,同时骑手状态释放为空闲;通过 WebSocket 实时接收新订单。

■ 功能模块与用户的对应一览表:

功能模块	顾客	商家	骑手
用户注册	✓	✓	✓
用户登录	✓	✓	✓
JWT 鉴权与 RBAC	✓	✓	✓
商家列表查询	✓		
菜品浏览(过滤已下架/已删除)	✓		
收货地址 CRUD(软删除)	✓		
购物车下单(原子事务)	✓		
历史订单查询	✓		
取消待处理订单	✓		
菜品 CRUD(添加/编辑/软删除)		✓	
菜品上下架		✓	
订单列表与状态筛选		✓	
接单/拒单操作		✓	
订单查询	✓	✓	✓
可抢订单列表			✓
主动抢单			✓
我的配送订单			✓
确认送达			✓
WebSocket 实时推送接收		✓	✓

3 外部接口需求

3.1 用户接口

■ 用户输入数据:

用户类型	输入数据	接口类型
顾客	用户名、密码（注册/登录）；收货地址（地址文本+电话号码）；菜品选择与数量；订单取消确认	Web 表单、按钮操作、下拉选择、数量调整
商家	用户名、密码（注册/登录）；菜品信息（名称+价格）；接单/拒单确认	Web 表单、按钮操作、下拉选择
骑手	用户名、密码（注册/登录）；抢单确认；送达确认	Web 表单、按钮操作

■ 系统输出数据:

输出类型	数据内容	展现形式
认证结果	JWT Token (access_token)、用户 ID、角色类型、用户名	API Response JSON
商家列表	商家 ID、商家名称、在售菜品数量 卡片网格	卡片网格
菜品列表	菜品 ID、名称、价格、上架状态 列表卡片	列表卡片
订单信息	订单 ID、顾客名、商家名、菜品摘要、总额、状态中文名、时间	卡片/表格列表
实时推送	新订单提醒（商家）、新任务提醒（骑手）、订单取消提醒（商家）	浏览器通知弹窗
错误提示	业务校验失败原因（如“仅待处理订单可取消”、“该订单已被其他骑手抢走”）	Element Plus Message / 弹窗

3.2 硬件接口

本系统为纯 B/S 架构 Web 应用，不直接与硬件设备交互。系统运行在通用 x86 架构 PC 服务器上，用户通过标准浏览器（Chrome / Edge）访问。

3.3 软件接口

接口名	描述	技术实现
RESTful API 接口	前后端数据通信接口，统一 {code, message, data} 响应格式	HTTP/HTTPS, FastAPI 框架
WebSocket API	服务端向指定商家/骑手实时推送订单状态变更消息	WS 协议, websockets 库
Swagger UI	API 接口文档自动生成，支持在线交互式测试	FastAPI 内置 /docs
数据库接口	ORM 抽象层，避免手写 SQL 拼接	SQLAlchemy 2.0 异步模式
JWT 认证接口	Token 签发与验证，客户端通过 HTTPBearer 方式传递	python-jose 库
Vite 代理接口	开发环境前端自动将 /api 请求代理到后端	Vite proxy 配置

内部组件间数据交换：

组件间	交换数据	交换方式
前端 → 后端	认证凭证、CRUD 请求参数、订单数据	Axios HTTP + JWT Bearer Header
后端 → 前端	JSON 响应数据、WebSocket 推送消息	HTTP Response + WS JSON Message
路由层 → 数据层	SQLAlchemy ORM 查询/插入/更新/删除	AsyncSession 异步查询
路由层 → WebSocket 管理器	推送消息对象	ws_manager.send_to_merchant() / send_to_courier()
中间件层 → 路由层	已认证的用户 ORM 实例	FastAPI Depends 依赖注入

3.4 通讯接口

通信方式	协议	默认端口	用途
HTTP REST API	HTTP/1.1	8001	常规 CRUD 操作、认证请求
WebSocket	WS	8001	实时订单状态推送（商家端、骑手端）
Vite Dev Proxy	HTTP/1.1	5173	前端开发服务器代理 /api 请求到后端
MySQL TCP 连接	MySQL Wire Protocol	3306	生产环境数据库连接（开发环境使用本地 SQLite 文件）

WebSocket 消息格式:

<pre>{ "type": "new_order order_cancelled order_assigned", "title": "消息标题（含 Emoji 图标）", "body": "消息正文", "order_id": 123, "timestamp": "2026-06-07T12:00:00" }</pre>

WebSocket 消息推送场景:

消息类型	触发条件	推送目标
new_order	顾客下单成功	对应商家
order_cancelled	顾客取消订单	对应商家
order_accepted	商家接单	对应骑手

WebSocket 连接鉴权:

连接端点: `ws://host:8001/api/merchant/ws/{role}/{user\_id}?token={jwt\_token}`

鉴权方式: 通过 Query String 传递 JWT Token, 服务端调用 `parse\_token\_optional()` 解析校验 Token 验证失败 → 关闭连接, 返回 code=4003 (Token 无效或角色不匹配)

前端断线自动重连: 5 秒后自动重新建立 WebSocket 连接

安全性:

JWT Token 有效期 24 小时, 过期需重新登录

WebSocket 连接需附带有效 Token, 中间人无法伪造

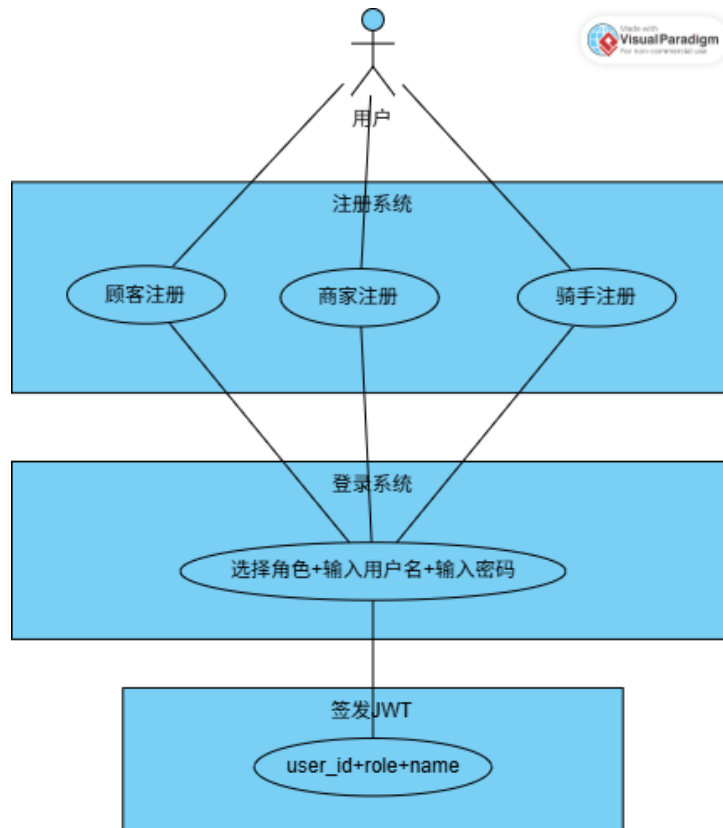
前端 401 拦截器在 Token 过期时自动清除登录状态并重定向

4 实现方式

4.1 多端统一认证功能

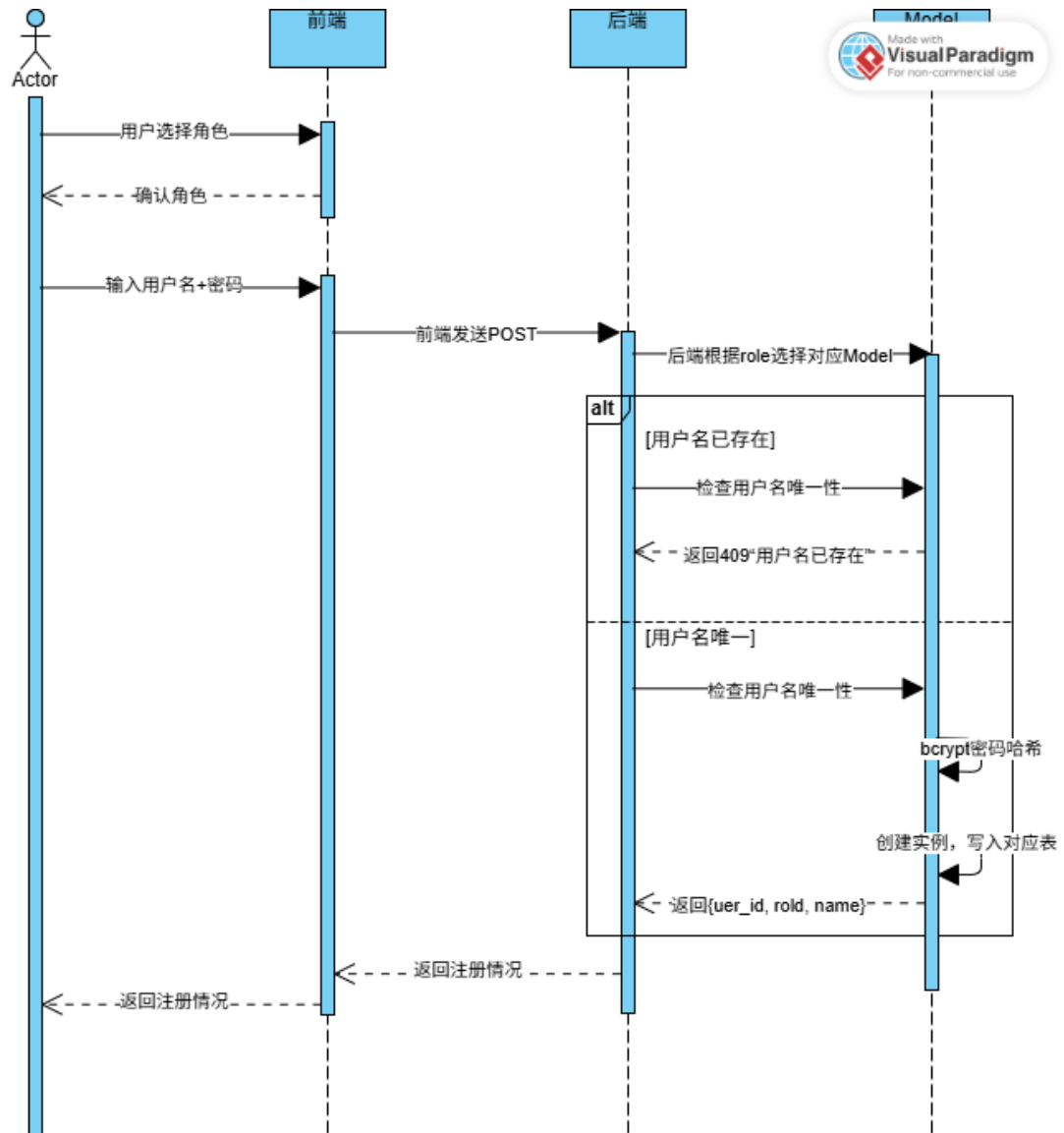
4.1.1 功能概要

本功能模块实现顾客、商家、骑手三种角色在同一登录页面的统一注册与登录。系统根据用户选择的角色类型，将账户信息写入对应的数据库表（customers / merchants / couriers），并签发包含用户身份信息的 JWT Token。后端通过角色依赖注入确保各端 API 只能被对应角色的用户访问。

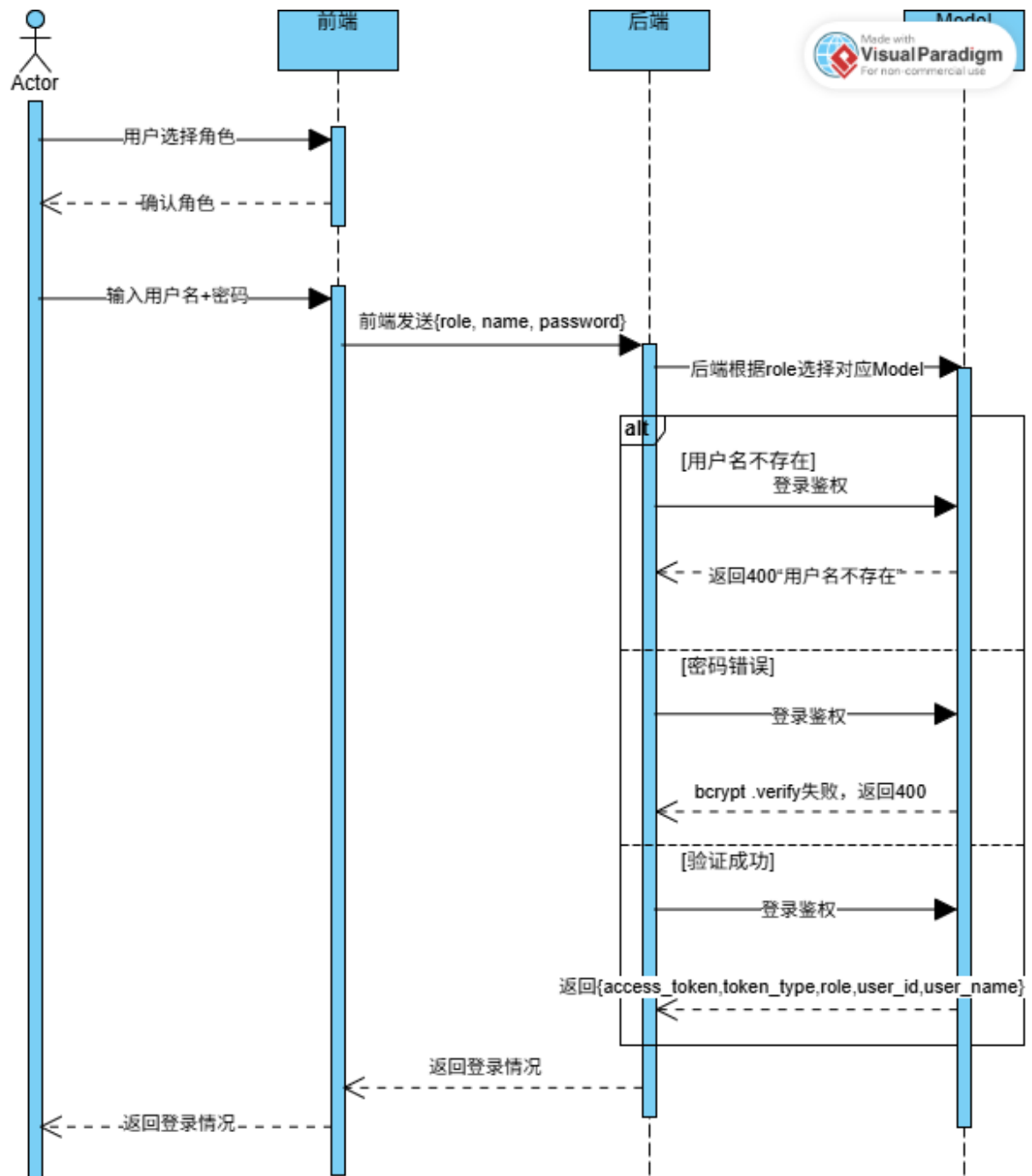


4.1.2 注册/登录流程

注册流程:



登录鉴权流程：



4.1.3 相关功能

组件名	说明	关系
所有业务路由	各端业务路由依赖认证中间件注入当前用户实例	所有端点均 `Depends(get_current_xxx)`
前端路由守卫	基于 Pinia auth store 的 token + role 控制页面访问	角色校验 + 未登录重定向到 /
WebSocket 端点	WS 连接时需通过 `parse_token_optional()` 解析 Token	Token 放在 Query String 中传递

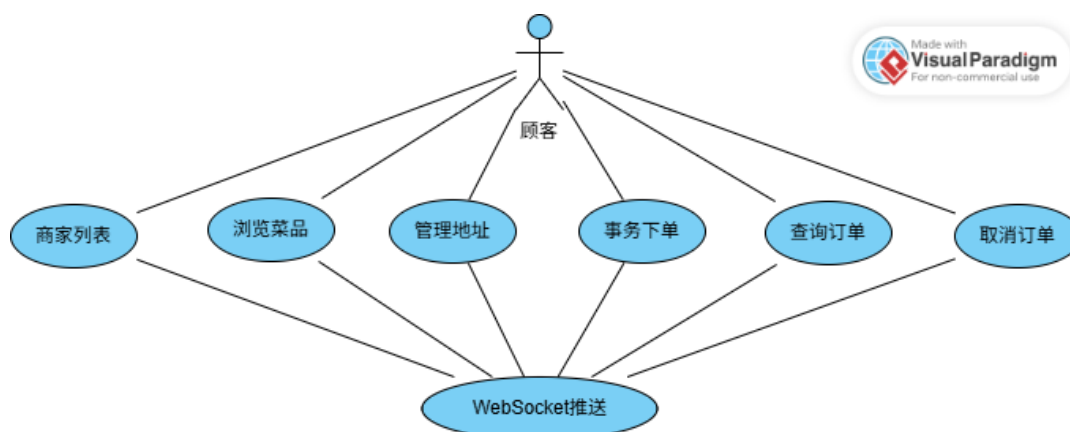
4.1.4 用户接口

接口端点	方法	输入	输出
`/api/auth/register`	POST	`{role: "customer"   "merchant"   "courier", name, password}`	`{code: 200, message: "注册成功", data: {user_id, name, role}}`
`/api/auth/login`	POST	`{role: "customer"   "merchant"   "courier", name, password}`	`{code: 200, message: "登录成功", data: {access_token, token_type, role, user_id, user_name}}`

4.2 顾客下单功能

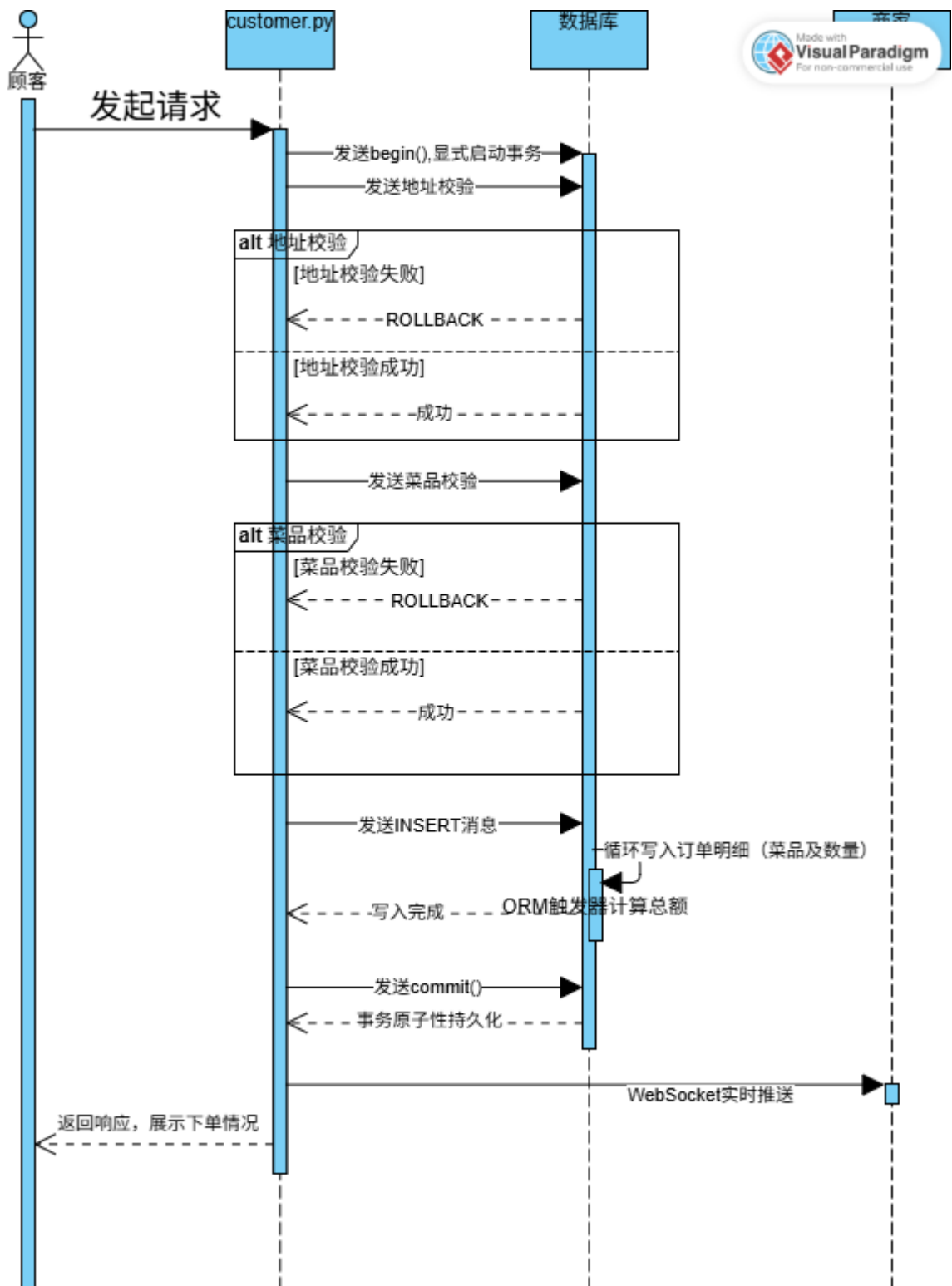
4.2.1 功能概要

顾客端实现从“浏览商家 → 选择菜品 → 管理地址 → 提交订单 → 查看订单 → 取消订单”的完整点餐流程。核心下单接口采用显式异步事务（`async with db.begin()`）保证原子性：任何校验失败或运行时异常将触发自动 ROLLBACK，杜绝产生僵尸订单（仅有 Orders 主记录而无 OrderDish 明细的脏数据）。

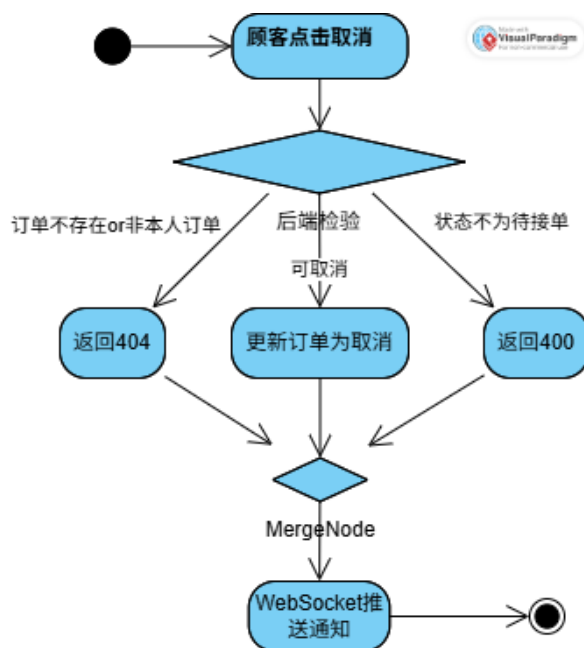


4.2.2 功能详细

事务下单流程：



订单取消流程:



4.2.3 相关功能

组件名	说明	关系
WebSocket 管理器	下单成功后推送 new_order 提醒给对应商家；取消后推送 order_cancelled	<code>`ws_manager.send_to_merchant()`</code>
ORM 触发器	下单明细写入后自动触发总额计算	OrderDish after_insert 事件
商家路由	商家接单/拒单依赖于订单状态=0	状态机联动

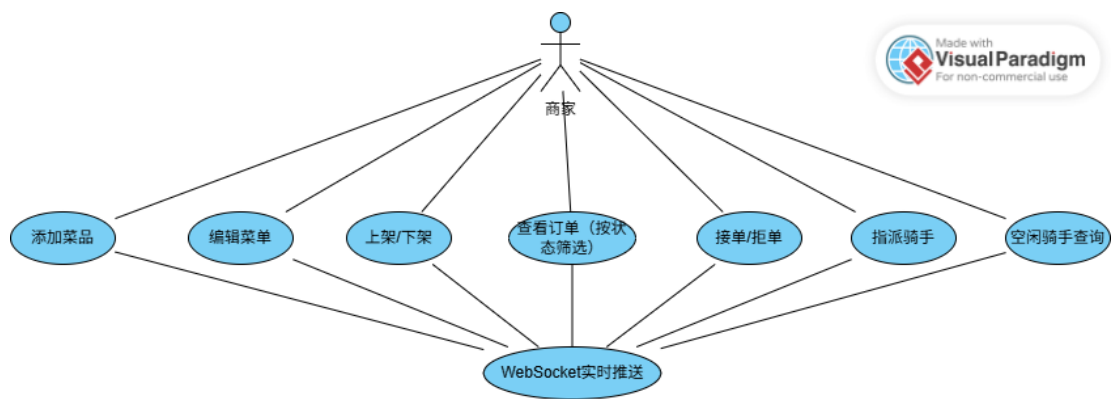
4.2.4 用户接口

接口端点	方法	描述	鉴权角色
<code>/api/customer/merchants</code>	GET	获取有在售菜品的商家列表（含菜品数量统计）	顾客
<code>/api/customer/dishes?merchant_id=</code>	GET	浏览菜品（仅展示 <code>is_deleted=0</code> 且 <code>is_active=1</code> ）	顾客
<code>/api/customer/addresses</code>	GET	获取当前顾客未删除的地址列表	顾客
<code>/api/customer/addresses</code>	POST	新增收货地址	顾客
<code>/api/customer/addresses/{id}</code>	DELETE	软删除地址（ <code>is_deleted=1</code> ，历史订单不受影响）	顾客
<code>/api/customer/order</code>	POST	事务下单（原子化：地址校验→菜品校验→插入订单→插入明细→自动计算总额）	顾客
<code>/api/customer/orders</code>	GET	查看历史订单（含菜品详情拼装，按时间倒序）	顾客
<code>/api/customer/order/{id}/cancel</code>	PUT	取消待处理订单（0→4，状态机限制）	顾客

4.3 商家管理功能

4.3.1 功能概要

商家端提供菜品全生命周期管理（添加/编辑/软删除/上下架）和订单处理功能（查看/接单/拒单）。支持 WebSocket 实时接收新订单推送通知以完成派单。

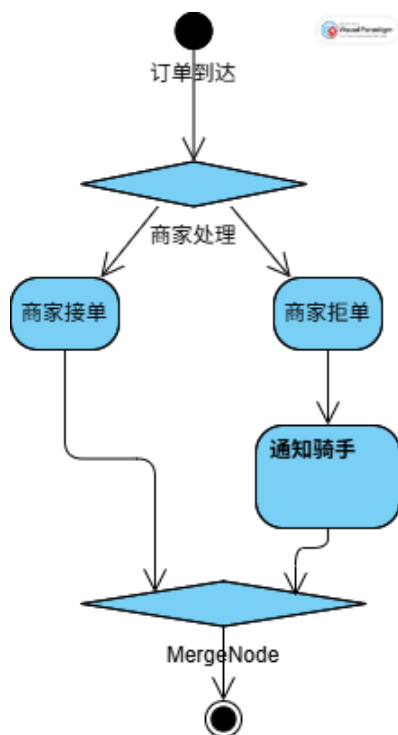


4.3.2 功能详细

菜品状态矩阵:

is_deleted	is_active	状态	顾客可见	商家可见
0	1	正常上架	可见	可见（可操作）
0	0	正常下架	不可见	可见（可重新上架）
1	1	已删除	不可见	可见（不可恢复）
1	0	已删除	不可见	可见（不可恢复）

订单处理流程:



4.3.3 相关功能

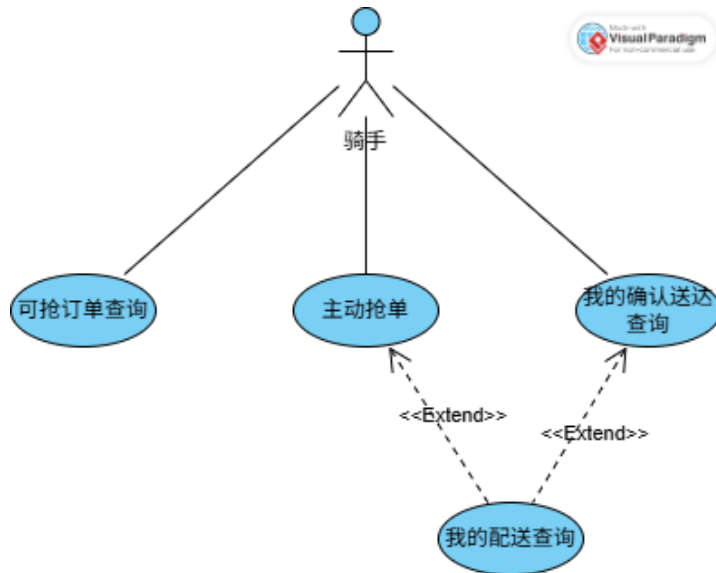
组件名	说明	关系
WebSocket 管理器	实时接收新订单推送 (new_order); 接单后提醒骑手 (oredr_accepted)	<code>`send_to_merchant()`</code> / <code>`send_to_courier()`</code>
骑手模块	骑手状态变更 (空闲↔忙碌) 与订单状态同步更新	抢单时原子更新 <code>courier.status</code>
认证模块	商家角色鉴权, 校验订单归属	<code>`Depends(get_current_merchant)`</code>

4.3.4 用户接口

4.4 骑手配送功能

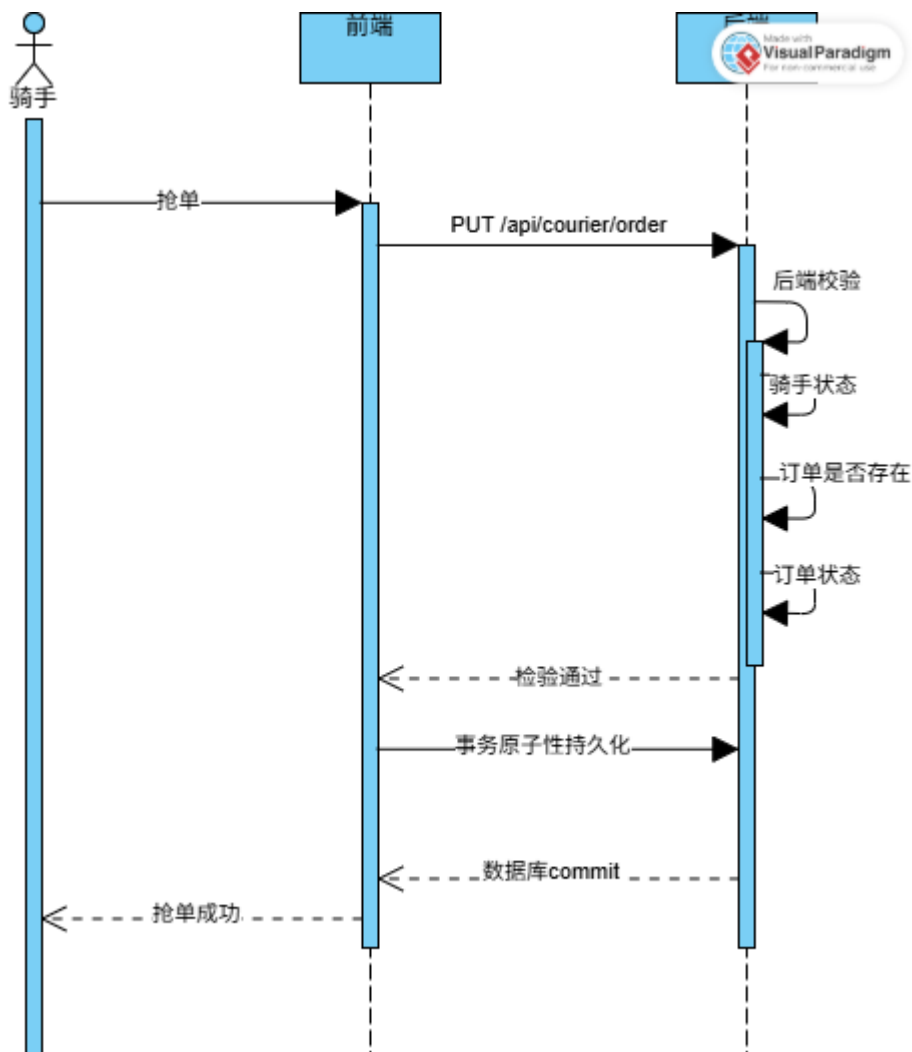
4.4.1 功能概要

骑手端实现主动抢单和配送确认功能。骑手查看“商家已接单但无骑手配送”的订单列表，主动发起抢单；抢单成功后骑手状态变为“忙碌”；配送完成后确认送达，骑手状态恢复为“空闲”。



4.4.2 功能详细

抢单流程：



补充事项:

骑手状态:

无配送订单则为空闲状态, 抢单则转为忙碌状态。从忙碌状态转空闲状态需确认送达订单。

4.4.3 用户接口

接口端点	方法	描述	鉴权角色
`/api/courier/available-orders`	GET	查看可抢订单 (status=1 且 courier_id IS NULL, 按时间倒序)	骑手
`/api/courier/order/{id}/pickup`	PUT	抢单 (1→2, 骑手 0→1, 原子操作)	骑手
`/api/courier/orders`	GET	查看我的配送订单 (courier_id=当前骑手ID)	骑手
`/api/courier/order/{id}/deliver`	PUT	确认送达 (2→3, 骑手 1→0, 原子操作)	骑手

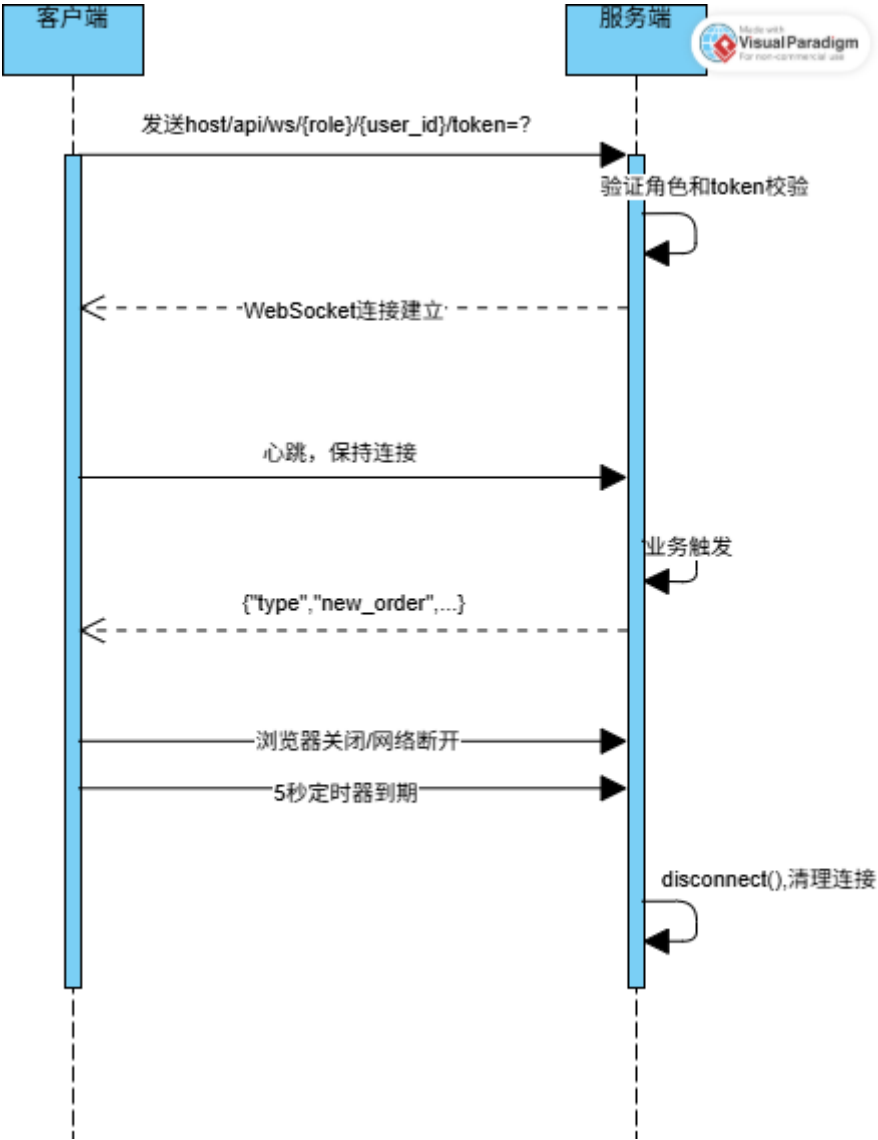
4.5 WebSocket 实时推送功能

4.5.1 功能概要

WebSocket 模块为商家端和骑手端提供实时消息推送服务。当订单状态发生变化时 (新订单、订

单取消), 服务端通过 WebSocket 长连接向目标用户实时推送 JSON 格式通知消息, 用户无需手动刷新页面即可获得最新订单动态。

4.5.2 功能详细



4.5.3 相关功能

组件名	说明	关系
认证模块	WS 连接时的 Token 鉴权	<code>parse_token_optional(token)</code>
顾客路由	下单成功/取消订单后触发 WS 推送	<code>ws_manager.send_to_merchant()</code>
商家路由	接单后触发 WS 推送	<code>ws_manager.send_to_courier()</code>

4.5.4 用户接口

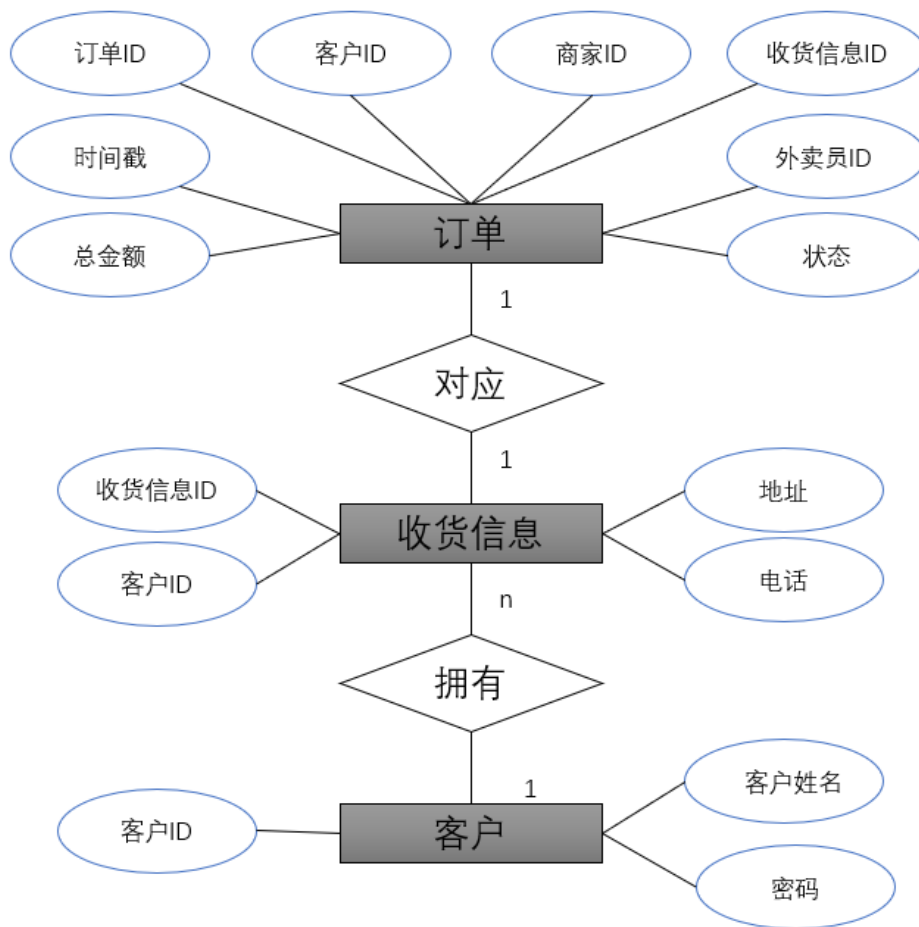
接口端点	方法	描述	鉴权方式
<code>~/api/ws/{role}/{user_id}?token=</code>	WebSoocket	商家/骑手实时消息接收与心跳保持	JWT Token(Query String)

4.6 数据库层功能

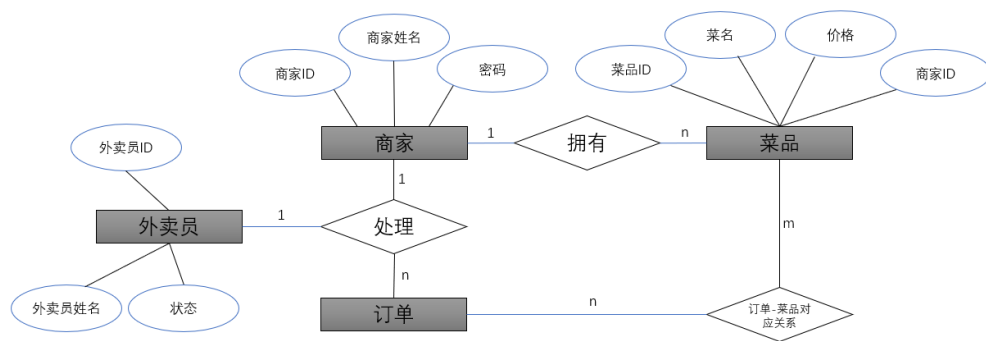
4.6.1 功能概要

数据库层采用 SQLAlchemy 2.0 异步 ORM 框架, 共设计 7 张核心业务表, 通过声明式模型映射数据库表结构。实现 ORM 事件触发器自动计算订单总额。

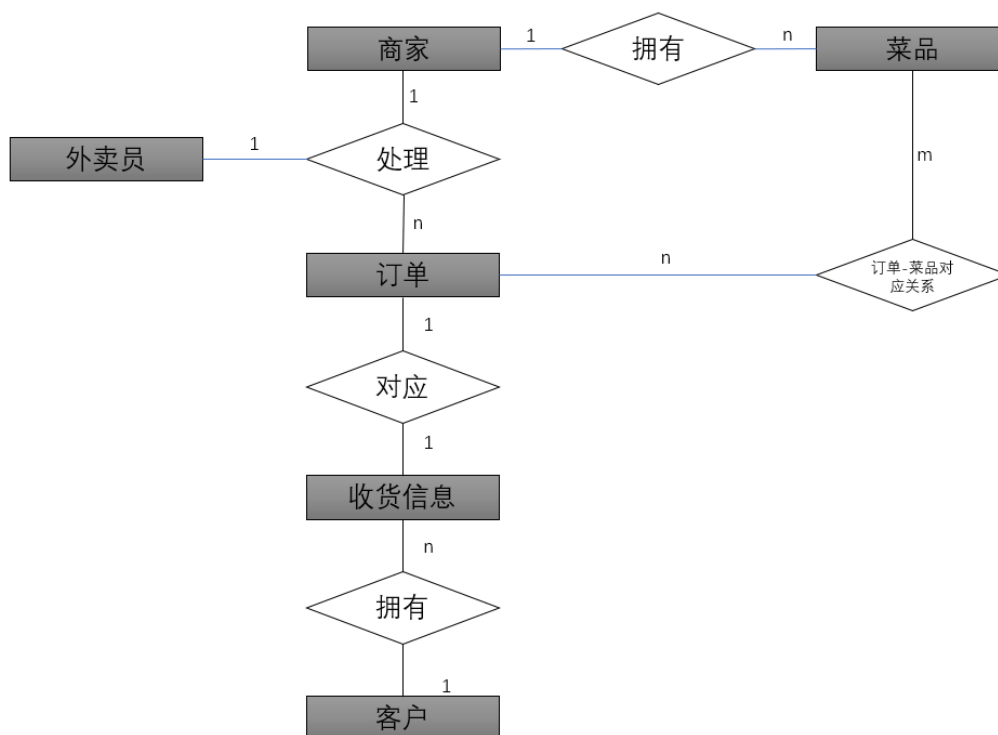
客户子 ER 图



商家子 ER 图：



总 ER 图：



4.6.2 功能详细

核心业务表:

实体转换说明:

编号	实体	实体转换说明与转换形成的关系
1	客户	Customer(客户 ID, 客户姓名, 密码)
2	外卖员	Courier(外卖员 ID, 外卖员姓名, 状态), 状态为空闲或忙碌。无配送任务则是空闲, 否则忙碌
3	商家	Merchant(商家 ID, 商户名字, 密码)
4	菜品	Dish(菜品 ID, 菜名, 价格, 商家 ID)
5	收货信息	Shipping(收货信息 ID, 客户 ID, 地址, 电话)
6	订单	Order(订单 ID, 客户 ID, 商家 ID, 收货信息 ID, 外卖员 ID, 时间戳, 总金额, 状态) 状态有待处理、已接单、配送中、已送达和已完成 总金额由订单-菜品对应关系计算得出

联系转换说明:

编号	联系	联系转换说明与转换形成的关系
1	订单-菜品对应关系 (m:n)	Order_dish(订单菜品 ID, 订单 ID, 菜品 ID, 数量)

ORM 事件触发器:

OrderDish 变更 (INSERT/UPDATE/DELETE) 会触发 SQLAlchemy ORM Event。

4.6.3 相关功能

组件名	说明	关系
所有路由模块	所有数据读写均通过 SQLAlchemy ORM AsyncSession 操作	依赖 `get_db()` 依赖注入获取会话
Config.py	数据库连接 URL 配置、引擎参数	`settings.DATABASE_URL`
Database.py	引擎初始化 (SQLite/MySQL 自适应)、会话工厂	应用启动时通过 lifespan 创建表

4.6.4 用户接口

数据库层不直接面向用户，通过 ORM 向服务层提供数据访问能力。开发阶段数据库表在应用启动时通过 ``Base.metadata.create_all`` 自动创建

5 非功能需求

5.1 权限需求

系统采用 RBAC (Role-Based Access Control) 权限模型，基于 JWT Token 中的 `role` 字段进行权限控制。

用户角色	可访问页面路由	可调用 API 前缀	权限边界
Customer	/customer	/api/auth/*, /api/customer/*	仅可操作本人订单和地址 (数据归属校验)
Merchant	/merchant	/api/auth/*, /api/merchant/*	仅可操作本店菜品和订单 (数据归属校验)
courier	/courier	/api/auth/*, /api/courier/*	仅可操作本人配送订单

权限控制三层实现：

层级	实现方式
后端 API 层	每个端点通过 `Depends(get_current_xxx)` 声明角色依赖，Token 角色不匹配返回 403
前端路由层	`beforeEach` 路由守卫校验 `meta.role` 与 Pinia auth store 中的 role 是否一致
WebSocket 层	连接时通过 `parse_token_optional()` 解析 Token 并校验 role 与 user_id 匹配

5.2 兼容互换性

【没有特别要求】

本系统为全新开发项目，不涉及对旧版本系统的兼容性需求。向下兼容性不是本次开发的范围。

5.3 安全性

5.3.1 标准符合性：

法律法规符合性：

系统采用安全认证、访问控制、数据完整性保护等技术措施，符合《中华人民共和国网络安全法》第 21 条（等级保护）。

用户密码经 bcrypt 加密存储，不保存任何明文敏感信息，符合《中华人民共和国网络安全法》第 40-43 条（个人信息保护）。

仅采集实现业务功能所必需的最少信息：用户名、密码哈希、地址、电话，符合《中华人民共和国个人信息保护法》第 6 条。

数据库采用 ORM 参数化查询，软删除机制保证数据可追溯与可恢复，符合《中华人民共和国数据安全法》第 27 条。

JWT + RBAC 三端鉴权、数据归属校验、路由守卫多层防护，符合《信息安全技术 个人信息安全规范》（GB/T 35273-2020）身份认证失败、失效的访问控制。

基准确认：

基准项	目标
身份认证	所有业务 API 端点必须在鉴权通过后才可访问
访问控制	用户仅能访问与其角色匹配的 API 端点，且仅能操作其自身数据
密码安全	禁止明文存储密码，密码不得以任何形式在日志、响应中泄露
会话安全	采用无状态 Token，设置合理的过期时间，支持前端自动失效处理
最小权限原则	各角色仅拥有完成自身业务所必需的最小权限集
安全配置	JWT 密钥、数据库连接串等敏感配置外部化，不可硬编码在源码中

用户在安全方面的需求：

本节从三端用户视角出发，陈述用户对系统安全性的核心诉求。

用户角色	安全需求	需求说明
顾客	账户密码安全	我的登录密码不能被明文存储或泄露给任何人，包括系统管理员
	订单信息安全	我的收货地址、联系电话、订单记录仅应被我自己和对应订单的商家/骑手在必要范围内知晓
	操作不可抵赖	我提交的订单、取消的订单需要有明确的记录，不能被他人冒用我的身份操作
	数据自主控制	我可以主动删除不再使用的收货地址，删除后不应影响我已有的历史订单查询
商家	账户与菜品安全	只有我自己可以管理我的菜品信息（添加/修改/下架/删除），其他商家或顾客无权修改
	订单处理权限	只有我本人可以对我店铺的订单进行接单/拒单操作
	实时消息可信	我收到的 WebSocket 新订单推送必须来源于系统的合法下单行为，不能被伪造成虚假订单
	经营数据隐私	我的菜品定价、订单流水等经营数据不应被其他商家查看
骑手	账户与配送安全	只有我本人可以确认配送完成，他人不可冒用我的身份完成配送
	抢单公平性	所有空闲骑手应享有同等的抢单机会，系统不应预设优先级或偏向特定骑手
	配送记录准确	我的配送记录（抢了哪些单、是否已完成）应完整准确，作为工作凭证

任务通知可信	我收到的 WebSocket 通知应来源于商家的接单操作
--------	------------------------------

5.3.2 安全设施需求:

No	危险/损失场景	影响范围	严重程度	触发条件
01	用户密码明文泄露	所有用户账户安全	严重	密码以明文存储; 日志/API 响应中包含密码字段
02	JWT 密钥泄露导致 Token 伪造	整个认证体系失效	严重	JWT_SECRET 硬编码在源码中且源码对外泄露; `.env` 文件被提交至公开仓库
03	订单数据不一致 (僵尸订单)	数据库完整性	严重	下单过程中途异常但部分数据已写入 (有 orders 主记录无 order_dishes 明细)
04	越权操作导致数据破坏	特定用户的订单/菜品/地址	较严重	用户通过修改请求 URL 中的 ID 参数尝试访问他人资源
05	竞态条件导致重复派单	同一订单被多个骑手同时抢到	较严重	高并发场景下多个骑手同时对同一订单发起抢单请求
06	误删除导致数据不可恢复	菜品/地址数据	中等	商家或顾客在未确认的情况下执行删除操作
07	WebSocket 中间人攻击	实时推送消息被篡改或窃听	中等	WebSocket 连接未加密 (ws://), 攻击者在同一网络中进行数据包捕获
08	Token 长期有效导致被盗用	已签发的旧 Token 被恶意使用	中等	Token 没有过期时间或过期时间过长

5.3.3 安全保护措施与动作:

针对上述危险, 系统必须实施以下安全保护措施:

No	保护措施	对应危险	实现要求	验证方法
01	bcrypt 密码单向哈希	01	所有用户密码在写入数据库前必须经 bcrypt 算法单向哈希处理; 系统中任何位置不得存储、传输、打印、记录明文密码; 密码比对必须使用 `verify_password()` 函数, 不得明文对比	检查数据库 password_hash 字段: 所有值均以 `\$2b\$` 开头 (bcrypt 特征前缀); grep 搜索日志/响应中无明文密码
02	JWT 密钥外部化与隔离	02	`JWT_SECRET` 必须通过环境变量或 `.env` 文件注入, 不得硬编码在源码中; `.env` 文件必须加入 `.gitignore` 不纳入版本控制; 生产环境 JWT_SECRET 长度不得少于 64 位随机字符	`JWT_SECRET` 从 `Settings` 类中读取环境变量;
03	原子事务保护	03	下单接口必须使用 `async with db.begin()` 开启显式事务边界; 订单主记录 INSERT 和订单明细 INSERT 必须在同一事务内完成; 任何校验失败或运行时异常触发自动 ROLLBACK; 事务提交前不得进行 WebSocket 推送 (避免推送了但数据未持久化)	模拟下单异常 (如无效菜品 ID), 验证数据库中无残留的 orders 记录
04	数据归属校验	04	所有涉及特定资源 (订单/菜品/地址) 的操作, 在执行业务逻辑前必须校验该资源归属于当前认证用户; 商家只能操作 `merchant_id` 为本商家的资源; 顾客只能操作 `customer_id` 为本人或 `shipping_id` 对应的地址为本人的资源;	编写测试用例: 用户 A 的 Token 尝试操作用户 B 的资源, 验证返回 403

			骑手只能操作 `courier_id` 为本人或尚无骑手的资源	
05	业务规则前置校验 + 数据库约束	05	抢单操作须在代码层先做前置条件校验（`order.status==1` 且 `order.courier_id IS NULL` 且 `courier.status==0`）；状态更新与骑手状态变更应在同一事务中完成；建议未来版本添加数据库行级锁（`SELECT ... FOR UPDATE`）或 Redis 分布式锁彻底消除竞态条件	当前版本通过业务校验覆盖常规场景
06	关键操作确认对话框	06	所有不可逆或具有破坏性的操作（取消订单、删除地址、删除菜品、确认送达、切换商家清空购物车）必须在用户界面弹出 `ElMessageBox` 确认对话框，用户明确确认后方可执行	手动操作验证每个关键操作均有弹窗确认
07	WebSocket Token 鉴权	07	WebSocket 连接请求必须携带有效 JWT Token（通过 Query String `token` 参数传递）；服务端必须在 `accept()` 前完成 Token 校验和角色匹配；校验失败必须主动关闭连接并返回 code=4003	使用不带 Token 或带无效 Token 的 WebSocket 请求尝试连接，验证连接被拒绝
08	Token 过期机制	08	JWT Token 必须设置合理的过期时间（默认 24 小时，即 `JWT_EXPIRE_MINUTES=1440`）；前端须在接收到 401 响应时自动清除本地 Token 并重定向到登录页	等待 Token 过期后发起 API 请求，验证返回 401 且前端自动跳转登录页

数据传输安全：

传输场景	安全需求
HTTP API 请求/响应	所有 API 通信应通过加密信道传输，防止中间人窃听或篡改
WebSocket 实时通信	WebSocket 消息应通过加密信道传输
数据库连接	数据库连接应支持加密传输
JWT Token 传递	Token 在 HTTP Header 中以 `Authorization: Bearer <token>` 形式传递，不应出现在 URL 参数中

数据存储安全：

存储对象	安全需求
用户密码	密钥文件权限应限制为仅应用运行用户可读
JWT Secret	密钥文件权限应限制为仅应用运行用户可读
数据库文件（SQLite）	数据库文件应设置适当的操作系统文件权限
日志文件	日志中不得记录密码、JWT Secret、完整的 JWT Token 等敏感信息

数据备份安全：

备份对象	备份策略	备份要求
数据库全量备份	每日全量备份	MySQL 生产环境建议使用 `mysqldump` 每日凌晨自动备份，保留最近 7 天的日备份和最近 4 周的周备份；SQLite 开发环境建议每次重大变更前手动复制 `takeaway.db` 文件
订单数据	包含在全量备份	订单数据为核心业务数据，备份时必须保证数据一致性（建议在数据库低负载时段执行备份，或使用事务一致性快照）
配置文件	每次修改后即时备份	`.env`、`config.py`、Nginx 配置等关键配置文件变更后应

		立即备份
备份验证	每月至少一次	定期从备份文件中恢复数据到测试环境，验证备份的完整性和可用性；如果备份文件损坏而无人知晓，备份策略形同虚设
备份存储	异地/异介质存储	备份文件应存储在不同于主数据库的物理位置或存储介质上；建议使用云存储（如阿里云 OSS、AWS S3）或内网 NAS 作为备份目标

5.3.4 其他安全型需求：

用户身份确认与授权需求：

1. 用户密码长度不得少于 4 个字符
2. 注册时选择的角色决定了用户的身份类别，注册后不可更改角色
3. 系统假定每个用户名只注册一种角色（同一用户名可在不同角色表中同时存在，但视为不同身份的独立账户）
4. 同一用户可在多个浏览器/设备上同时登录，每个设备持有独立 Token
5. 用户可通过点击「退出」按钮主动清除本地 Token，退出当前登录状态

系统安全性与完整性策略：

1. 订单状态必须严格按照预定义的状态机规则流转，不允许跳过中间状态或回退状态
2. 订单一旦创建，其 `customer_id`、`merchant_id` 不可变更
3. 软删除操作（`is_deleted=1`）一旦执行，代码层不提供“恢复未删除”操作
4. 订单总额由 ORM 事件自动触发计算，不允许通过 API 手动修改 `total` 字段
5. 推送消息的 `type/order_id/timestamp` 字段由服务端生成并签名，客户端仅展示，不信任客户端传回的数据

隐私保护需求：

1. 系统仅采集完成外卖交易流程所必需的个人信息（用户名、收货地址、联系电话），不采集身份证号、银行卡号、地理位置等非必要信息
2. 顾客的收货地址（含详细地址文本和电话号码）仅在订单详情中对处理该订单的商家和骑手可见；其他顾客、其他商家、其他骑手无权查看
3. 不同角色表之间的数据在物理上隔离（独立数据表），不存在跨角色数据泄露的风险
4. 当前版本用户名可能包含真实姓名（`name` 字段），在订单详情中对相关方可见（如商家看到顾客名、骑手看到商家名）
5. 用户有权删除其个人数据（地址软删除）；若将来需要彻底清除用户数据（账户注销），应提供数据匿名化或物理删除机制

5.4 健壮性

措施	实现说明
事务原子性	下单操作使用 <code>async with db.begin()</code> 显式事务边界，任何校验失败抛出 <code>HTTPException</code> （自动 <code>ROLLBACK</code> ），避免产生僵尸订单
WebSocket 自动重连	前端 <code>ws.onclose</code> 事件触发 5 秒后自动重连，网络短暂中断自动恢复
<code>HTTPException</code> 透传	下单接口中 <code>HTTPException</code> 异常直接透传
数据库连接池	MySQL 模式下配置 <code>pool_size=20</code> 、 <code>max_overflow=10</code> 、 <code>pool_pre_ping=True</code> ，防止连接池耗尽和失效连接复用
SQLite 特殊处理	SQLite 模式下设置 <code>check_same_thread=False</code> ，适配异步场景
软删除机制	菜品和地址采用软删除（ <code>is_deleted=1</code> ），删除操作仅修改标记位，历史订单的外键引用不会断裂
前端错误兜底	所有 API 调用使用 <code>try/catch</code> 包裹， <code>catch</code> 块静默处理（错误已在 <code>Axios</code> 拦截

	器中通过 ElMessage 提示)
--	--------------------

容错等级:

- 成熟性: 核心 API 经过 22 个端点逐一测试, 常规操作无预期故障
- 障害兼容性: 单个 API 异常不影响其他功能模块, 前端 catch 块保证页面不崩溃
- 恢复性: SQLite 文件数据库无需额外恢复操作; MySQL 模式下连接池自动重连

5.5 使用性 (操作性)

1. 三端统一登录界面:
同一页面通过角色选择按钮 (顾客/商家/骑手) 支持三种角色登录, 切换角色自动清空表单, 登录/注册状态一键切换
2. 响应式设计:
顾客端和骑手端: `max-width: 600px` 居中布局 (模拟移动端体验); 商家端: `max-width: 1100px` 宽表格布局 (PC 工作台体验)
3. 实时反馈:
商家端和骑手端通过 WebSocket 实时接收推送通知 (ElNotification 弹窗 + 新订单计数 badge)
4. 表单双重校验:
前端: Element Plus 表单规则验证 (必填、长度等); 后端: Pydantic 数据校验 (类型、范围、正则匹配)
5. 友好错误提示:
错误响应通过 Element Plus Message 组件展示后端返回的具体原因 (如“当前订单状态为「已接单」, 无法取消”)
6. 操作确认:
点击菜品自动加入购物车 (带 + 按钮动画), 购物车数量实时显示 badge, 切换商家自动清空购物车并提示
7. 视觉层次:
订单按状态以不同颜色的左边框区分 (待处理/已接单/派送中/已完成/已取消/已拒单), 看出订单状态

5.6 效率性（性能）

本项目为最小功能的实验性数据库软件，定位于中小型餐饮商家的外卖管理原型系统，不期望达到生产级性能表现。以下各项性能指标为实验环境下（单用户、本地部署、SQLite 数据库）的参考值，不构成对生产环境的性能承诺。

■ 硬盘容量：

本系统的磁盘占用主要由四部分构成：后端代码与依赖包、前端代码与依赖包（含构建产物）、数据库文件（SQLite）、日志文件。

1. 初始磁盘容量：约 200MB，后端 Python 依赖包（约 120 MB）+ 前端 node_modules（约 50 MB）+ 源代码（约 2 MB）+ 初始 SQLite 空数据库（约 4 KB）
2. 最大磁盘容量：约 700MB 以内，在初始容量的基础上，预期数据增长：SQLite 数据库文件（随订单、菜品、地址数据累积，预估单表 10,000 条记录时约增长至 50 MB）+ 前端构建产物 dist/（约 2 MB）+ 运行时日志文件（预估累计 50 MB 以内，需手动清理）+ 预留余量

从本产品运行环境的详细说明（参照 [2.2.1 硬件环境] (#221-硬件环境)）判断，推荐配置 512 GB SSD 远大于本系统最大容量需求，磁盘容量没有问题。

■ 内存使用量：

根据本产品的特性，内存占用主要指物理内存（RAM），系统运行期间存在两个主要进程：后端 Python 进程（FastAPI + Uvicorn + SQLAlchemy）和前端 Node.js 进程（Vite 开发服务器）。生产部署时前端为纯静态文件，无 Node.js 进程。

进程占用内存情况：

1. 后端：100MB 以内，单 worker 模式，开发环境 SQLite 模式。含 FastAPI 框架、SQLAlchemy ORM（`lazy="selectin"` 预加载策略）、WebSocket 长连接管理
2. 前端：150MB 以内，仅开发阶段需要。含 Vite 开发服务器、HMR 热更新、所有 node_modules 索引
3. 前端生产部署（Nginx）：无额外内存，生产部署时前端为纯静态 HTML/CSS/JS 文件，由 Nginx/Apache 托管，不产生额外的 Node.js 进程，内存占用可忽略不计
4. 合计（开发环境）：300MB 以内，后端 100 MB + 前端 150 MB
5. 合计（生产环境）：100MB 以内，仅后端 Server 进程

关于内存界限值及要求动作：

1. 正常运行界限值：300MB 以内（开发）/100MB 以内（生产），无特殊动作
2. 内存泄漏警告界限值：超过 300MB，检查 WebSocket 连接数是否异常增长（连接泄漏），检查 ORM Session 是否正确关闭
3. 最大容许内存：400MB 以内，重启服务进程释放内存

WebSocket 连接内存占用：

1. 单连接内存占用：约 1KB，每个 WebSocket 连接在 ConnectionManager 中存储为一个字典条目
2. 最大连接数（设计）：100 连接，中小型商家场景，预估同一时刻活跃连接数不超过 100
3. WebSocket 模块总内存：约 100KB，可忽略不计

关于虚拟内存：没有特别记述事项

■ 处理速度：

本项指业务请求的处理速度（从请求到达到响应返回的时间），以开发环境下单用户、本地 loopback（127.0.0.1）的基准测试为准。

业务操作涉及的主要处理内容和处理速度目标如下：

1. 用户登录：bcrypt 密码验证 + JWT Token 签发，目标 1 秒内
2. 菜品列表查询：数据库复合索引查询 + ORM 对象映射，目标 5000ms 内

3. 顾客下单：地址校验 + 逐个菜品校验（存在性/归属/上下架/删除状态）+ Orders 主记录 INSERT + OrderDish 明细循环 INSERT + ORM 触发器重算总额 + COMMIT + WebSocket 推送，目标 2 秒内
4. 订单列表查询：多表 JOIN（orders + customers + merchants + couriers + shipping_addresses + order_dishes + dishes），`lazy="selectin"` 预加载策略，目标 1 秒内
5. 商家接单/拒单：单条 UPDATE（status 字段）+ 数据归属校验，不涉及事务，单一状态改变，目标 500ms 内
6. 骑手抢单：骑手状态校验 + 订单状态校验 + 原子化 UPDATE（订单 status + courier_id + 骑手 status）+ COMMIT 竞态条件保护：当前版本未实现分布式锁，通过数据库行锁作为最后防线，目标 1 秒内
7. 确认送达：原子化 UPDATE（订单 status + 骑手 status）+ COMMIT 同抢单流程，两个字段在同一个事务中更新，目标 1 秒内
8. WebSocket 实时推送，业务操作完成后推送到客户端，客户端接收不含网络传输延迟（本地测试环境）；触发时机在 COMMIT 之后，保证推送数据的一致性，目标 1 秒以内

批量操作的性能界限值：

1. 下单菜品数：单次最多 20 个菜品，处理速度每个菜品约 50ms（校验 + 明细插入），20 个菜品约 1 秒额外耗时，总下单时间约 3 秒以内
2. 订单列表查询：单次最多返回 500 条，500 条带全部关联数据的订单查询约 2 秒以内
3. 菜品管理列表：单次最多返回 200 条，200 条菜品查询约 500 ms 以内

关于改造部分的性能要求：

没有特别记述的事项

■ 性能响应：

本项指从用户操作（点击/输入）到界面给出可感知反馈的时间，涉及前端渲染性能和前后端通信延迟。

1. 登录/注册按钮响应，2 秒以内提示操作结果
2. 菜品浏览首次加载，2 秒以内显示菜品列表
3. 购物车下单，3 秒以内提示下单结果
4. 商家菜品 CRUD 操作，1 秒以内显示操作结果
5. 订单状态切换（接单/拒单），2 秒以内提示抢单结果
6. Tab 切换，200 ms 以内完成切换
7. WebSocket 实时通知，消息到达后 1 秒以内弹出通知

异步操作设计（超过响应目标时的处理策略）

超时场景及对应处理策略：

1. 操作超过预期响应时间的 1.5 倍时，前端显示 loading 状态（按钮禁用 + 加载动画），禁止用户重复提交，防止产生重复订单
2. API 请求超过 10 秒无响应，Axios 请求超时自动中断，通过 ElMessage.error 提示用户“网络异常，请稍后重试”
3. WebSocket 连接中断，前端 `onclose` 事件触发，5 秒后自动重连；重连期间不影响用户继续浏览已加载数据

执行效率性：

具体评价维度与期望指标：

1. 同步处理，所有业务 API 采用同步（async/await）处理模式，FastAPI 异步 IO 避免了阻塞等待。单次 CRUD 操作（如地址新增、菜品编辑）在 SQLite 上的执行时间为数十毫秒级别
2. 事务处理，核心下单流程采用显式事务，原子化执行地址校验→菜品逐项校验→订单插入→详情插入→触发器计算总额→提交。事务内操作不作异步并行优化
3. ORM 查询优化，所有 relationship 采用 `lazy="selectin"` 策略：一次性批量加载所有关联对象，将 N+1 查询压缩为 2 次查询（主查询 + 关联批量查询）

4. 索引策略，针对高频查询条件建立复合索引，比如用户名唯一索引加速登录查找
5. WebSocket 推送，消息推送在业务事务 COMMIT 之后同步触发，不采用异步消息队列，保证“推送”必定反映已持久化的最新状态

资源效率性：

资源效率性表示系统是否能够有效利用所分配的资源。

1. CPU 利用率：本系统为典型的 IO 密集型 Web 应用，CPU 占用主要来自：bcrypt 密码哈希（登录时约 200-500ms 单次）、JSON 序列化/反序列化、JWT 签名/验证、ORM 对象映射。在空闲状态下 CPU 使用率接近 0%，单次请求处理期间 CPU 瞬时占用率不超过 10%（现代 CPU 单核）
2. 内存利用：ORM relationship `lazy="selectin"` 策略以少量额外内存开销换取大幅减少的数据库往返次数。WebSocket ConnectionManager 的连接字典为轻量结构，每个连接仅存储 WebSocket 对象引用
3. 数据库连接：开发环境（SQLite）不支持连接池，单连接串行执行。生产环境（MySQL）配置 `pool\_size=20, max\_overflow=10`，最大 30 个并发连接；每个业务处理完成后 Session 即刻释放归还连接池，避免连接泄漏
4. 静态资源：前端 SPA 构建产物（dist/）为纯静态文件，可被浏览器缓存，重复访问时不产生额外的服务端计算和网络传输开销
5. 日志资源：当前使用 uvicorn 标准输出日志，不写入文件（开发环境）。生产环境需配置日志轮转策略以防止日志文件无限增长消耗磁盘空间
6. 网络宽带：单次 API 响应数据量通常为 1-20 KB（JSON 格式），WebSocket 推送消息约 200 字节/条，对局域网或普通宽带无压力

资源释放策略：

相关资源与释放时机：

1. 数据库 Session，每次 HTTP 请求结束后由 FastAPI 依赖注入系统自动关闭
2. WebSocket 连接，客户端断开或服务端检测到异常时，从 ConnectionManager 字典中移除对应条目
3. Token 内存，JWT Token 无状态，服务端不存储会话信息，不存在会话累积导致的资源泄漏
4. 前端资源，页面切换时 Vue 组件自动销毁，WebSocket 连接和定时器正确清理

5.7 维护性

考虑到今后本软件从实验原型向正式产品演进的可能，对版本升级策略做如下规划：

■ 后端 Python 依赖包升级：

安全补丁升级：及时跟进 FastAPI、SQLAlchemy、python-jose 等关键依赖的安全更新，在测试环境验证后升级生产环境

主版本升级：FastAPI、SQLAlchemy 等核心框架的大版本升级前需评估 API 兼容性变更（Breaking Changes），在测试分支先行验证

新增依赖：优先选用社区活跃、文档完善的第三方包，避免引入无人维护的冷门依赖

■ 数据库 Schema 升级：

新增表/字段：初期原型阶段允许删除数据库重建；正式版本后需引入数据库迁移工具（如 Alembic），以版本化的迁移脚本管理 Schema 变更

字段类型变更：需编写数据迁移脚本（ALTER TABLE），并考虑已有数据的类型转换兼容性

回滚机制：每个迁移脚本需配套对应的降级（downgrade）脚本，确保升级失败时可回退到上一版本

■ 前端资源升级：

1. 前端静态文件升级：构建产物通过版本指纹化（Content Hash）命名，避免浏览器缓存导致新旧资源混用

2. 前端框架升级：Vue 3、Element Plus 等前端框架的主版本升级需在开发分支验证兼容性后再合并

今后考虑的版本升级机制：

1. API 版本化管理：当接口发生不兼容变更时，通过 URL 前缀（如 `/api/v1/` → `/api/v2/`）区分版本，给旧版客户端充足的迁移时间
2. 前后端分离部署的独立版本管理：前端和后端各自独立版本号，松耦合升级
3. 配置文件向前兼容：新增配置项在代码中提供合理的默认值，旧配置文件缺少新增配置项时系统仍可正常运行

计划配备的故障调查用日志输入输出功能：

为方便故障调查，系统拟配备以下日志功能：

1. HTTP 请求日志：输出内容有请求方法、URL 路径、响应状态码、处理耗时，用于快速定位是哪个 API 端点返回了异常状态码（4xx/5xx）
2. 业务异常日志：输出内容有业务校验失败的具体原因（如“菜品已下架”、“地址不属于当前用户”、“订单状态不允许该操作”），通过错误消息直接锁定业务规则违反的具体项，无需逐一排查校验逻辑
3. SQL 执行日志：输出内容有实际执行的 SQL 语句与参数值，当怀疑是数据库查询返回异常数据导致 Bug 时，开启 SQL 日志观察 ORM 生成的实际查询，排查 N+1 查询、索引未命中、关联数据缺失等问题
4. WebSocket 事件日志：连接建立/断开、推送消息类型、推送目标用户，当实时推送未到达客户端时，通过日志判断是消息未发出、发出了但连接不存在、还是连接存在但推送失败
5. 应用启动/关闭日志：数据库连接状态、引擎初始化结果、路由注册清单，服务启动失败时可快速判断是数据库连接问题还是代码加载异常
6. 未处理异常日志：Python traceback 堆栈信息（含文件名、行号、异常类型），定位运行时崩溃的直接原因和代码位置

生产环境日志管理预案：

日志按日轮转（Rotating File），保留最近 30 天的日志文件

超过保留期限的日志自动删除，防止磁盘空间耗尽

日志文件大小超过一定阈值（如 50 MB）时强制轮转

5.8 移植性

数据库迁移：开发环境使用 SQLite（零配置），生产环境切换 MySQL 仅需修改 `DATABASE\_URL` 配置项，ORM 层自动适配差异。

OS 兼容性：Python 3.10+ + Node.js 18+ 跨平台，源码无需修改即可在 Windows / macOS / Linux 上运行。

前端部署灵活性：生产构建产物（`npm run build` → `dist/`）为纯静态 HTML/CSS/JS 文件，可部署于 Nginx / Apache / Caddy 等任意 Web 服务器。

数据库方言适配：使用 SQLAlchemy ORM 抽象层，不依赖特定数据库方言；database.py 中针对 SQLite 做了 `check\_same\_thread` 特殊处理，其他数据库使用通用连接池配置。

移植策略：全程不依赖操作系统特有 API 或文件系统特性；密码哈希使用纯 Python 实现的 bcrypt（passlib 库），不依赖系统级加密模块。

5.9 用户文档

项目开发计划书：计划、分工、技术方案、日程、风险管理

需求与功能设计规格书：本文档

部署说明（README.md）：环境要求、安装步骤、配置说明、FAQ

数据库设计：models.py + 附录，7 张表 ORM 模型 + ER 图 + 数据字典

5.10 其他

没有特别记述的事项

6 使用和操作方法

6.1 环境设定

后端配置文件 (backend/app/config.py)

配置项: DATABASE_URL

默认值: sqlite+aiosqlite:///./takeaway.db

开发环境使用 SQLite 零配置; 生产环境改为 MySQL 连接串

配置项: JWT_SECRET

默认值: ~

生产环境务必修改为随机 64 位字符串

配置项: JWT_ALGORITHM

默认值: 1440 (24 小时)

Token 过期时间, 可根据安全需求调整

通过`.env`文件覆盖:

DATABASE_URL=mysql+asyncmy://user:pass@127.0.0.1:3306/takeaway

JWT_SECRET=your-production-random-secret-key

CORS_ORIGINS=["https://your-domain.com"]

前端配置 (frontend/vite.config.js)

Vite 开发服务器默认端口: 5173

API 代理配置: `/api` 请求自动转发到 `http://127.0.0.1:8001`

6.2 使用方法

6.2.1 安装依赖

后端:

```
cd backend
```

```
pip install -r requirements.txt
```

前端:

```
cd frontend
```

```
npm install
```

6.2.2 启动服务

启动后端:

```
cd backend
```

```
uvicorn app.main:app --host 0.0.0.0 --port 8001 -reload
```

启动成功显示:

```
`Uvicorn running on http://0.0.0.0:8001` 和 `Application startup complete.`
```

启动前端:

```
cd frontend
```

```
npm run dev
```

启动成功显示:

```
`VITE v5.x.x ready in xxx ms` 和 `Local: http://localhost:5173/`
```

6.2.3 验证服务

浏览器访问 `http://127.0.0.1:8001/` 显示:

```
{ "status": "ok", "service": "外卖管理系统 API", "version": "1.0.0" }
```

6.2.4 退出系统

后端：终端中按 ``Ctrl+C`` 停止 `uvicorn` 服务器

前端：终端中按 ``Ctrl+C`` 停止 `Vite` 开发服务器

用户退出登录：点击页面右上角「退出」按钮，清除 `Token` 并返回登录页

7 注意限制事项

7.1 制约 / 限制事项

1. 单商家单骑手模型：当前版本每个订单仅由一个商家处理、一个骑手配送，不支持多商家拼单或多骑手接力配送
2. 无支付集成：当前版本不包含在线支付功能，订单金额仅为展示用途，不涉及真实资金流转
3. 骑手抢单无分布式锁：高并发场景下可能存在竞态条件（同一订单被多个骑手同时抢到），当前版本通过数据库约束作为最后防线
4. 浏览器兼容性：建议使用 Chrome/Edge 最新版本，不支持 Internet Explorer
5. 数据库文件存储：开发环境 SQLite 数据库文件存储在项目目录下，需注意备份
6. 单服务器部署：系统设计为单实例部署，不支持分布式集群（WebSocket 状态存储在进程内存中）

7.2 假定事项

1. 开发阶段使用 SQLite：假定开发阶段无需安装配置 MySQL，使用 SQLite 零配置数据库
2. 用户规模：假定系统服务于中小型餐饮商家，并发用户在 100 以内
3. 单实例部署：假定系统部署在单台服务器上，不涉及分布式集群架构
4. 开发工具：假定开发人员使用 VS Code + Windows 10/11 环境
5. 一人一角色：假定每个用户只注册一种角色，同一用户名在不同角色表中可以重复（实际因三表独立可以存在）
6. 网络环境：假定用户在局域网或稳定的互联网环境中使用系统

7.3 特记事项

没有特别记述的事项

8 附录

8.1 业务规则

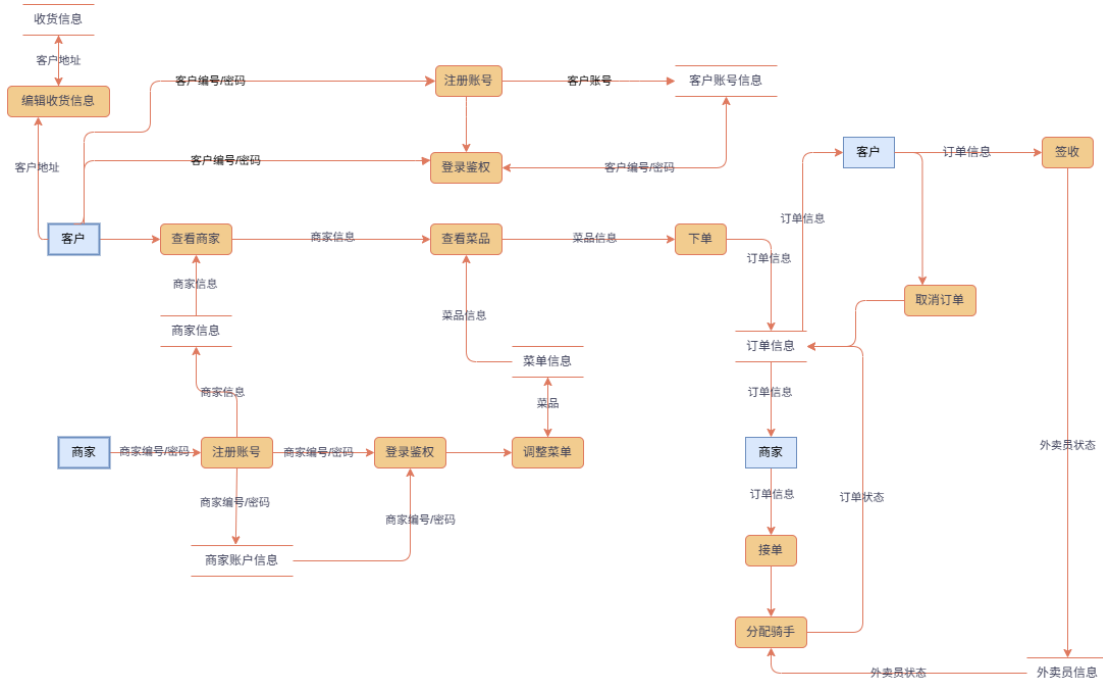
No	规则描述	涉及角色
01	订单仅在 待处理 状态时可被顾客取消	顾客
02	订单仅在 待处理 时可被商家接单	商家
03	订单仅在 待处理 时可被商家拒单	商家
04	仅空闲骑手可抢单	骑手
05	仅已接单且无骑手的订单可被抢	骑手
06	仅派送中且分配给自己的订单骑手可确认送达	骑手
07	顾客只能操作自己的订单 (customer_id 匹配) 和地址	顾客
08	商家只能操作自己店铺的菜品和订单 (merchant_id 匹配)	商家
09	骑手只能确认送达分配给自己的订单 (courier_id 匹配)	骑手
10	同一用户名不可在同一角色表中重复注册 (name UNIQUE 约束)	全部
11	菜品删除为软删除, 数据物理保留	商家
12	地址删除为软删除, 不影响历史订单	顾客
13	顾客下单时所有菜品必须属于同一个指定商家	顾客
14	订单总额由 ORM 触发器自动计算, 不允许手动修改	系统

8.2 术语表

术语	英文	说明
订单状态机	Order State Machine	订单状态有序流转规则: 0 待处理→1 已接单→2 派送中→3 已完成; 分支: 0→4 顾客已取消、0→5 商家已拒单
软删除	Soft Delete	通过标记字段 (is_deleted=1) 而非物理删除方式移除数据, 保留数据用于历史追溯
原子事务	Atomic Transaction	数据库事务保证多个操作要么全部成功提交 (COMMIT), 要么全部回滚 (ROLLBACK)
WebSocket 推送	WebSocket Push	服务端通过 TCP 长连接主动向指定客户端实时推送 JSON 消息
JWT Token	JSON Web Token	包含 user_id、role、name、过期时间的签名令牌, 用于无状态客户端认证
RBAC	Role-Based Access Control	基于角色的访问控制: 通过 payload 中的 role 字段决定用户可访问的 API 范围
ORM 事件	ORM Event	SQLAlchemy 提供的对象关系映射级事件钩子, 在 INSERT/UPDATE/DELETE 时触发自定义回调
依赖注入	Dependency Injection	FastAPI 的 Depends 机制, 用于自动注入数据库会话和认证用户实例
复合索引	Composite Index	多列组合的数据库索引, 加速多条件查询
连接池	Connection Pool	预创建并维护一组数据库连接, 复用连接减少开销

8.3 数据流图

该应用系统主要包括两类用户：客户和商家（外卖员内置）
系统整体的数据流图：



8.4 数据字典

下面列出数据流图的数据字典：

一、 数据结构

(1) 客户：系统的服务对象之一，通过客户编号和密码登录系统，可以进行下单、修改地址等操作。

输入数据流：

客户编号
密码
下单请求
取消订单请求
确认收货请求

输出数据流：

商家信息
菜品信息
订单状态信息

(2) 商家：可以通过商家编号和密码登陆系统，可以管理对应商家的菜品和订单，可以为订单派送指定外卖员。

输入数据流：

商家编号
密码
菜品管理请求
接单请求

输出数据流：

菜品信息
订单状态

二、 选取主要数据流进行说明，部分数据流过程重复，不再赘述

(1) 登录鉴权相关

数据流名：客户编号/密码

说明：客户登录认证信息

来源去向：客户—登录/鉴权

数据存储：客户数据存储

数据流名：客户鉴权成功

说明：客户身份验证结果

来源去向：登录/鉴权—客户界面

数据存储：客户数据存储

(2) 客户操作相关

数据流名：增删改查收货信息

说明：管理收货地址

来源去向：客户界面—收货信息

数据存储：收货信息数据存储、商家信息存储

(3) 菜品数据流

数据流名：增删改查菜品

说明：商家管理菜品信息

来源去向：商家界面—菜品

数据存储：商家数据存储、菜品

三、主要数据存储

(1) 客户：存储所有注册的客户信息

数据描述：顾客编号、密码、名称、年龄、性别

存取方式：按照顾客编号随机存取

（2）商家：存储所有入驻商家的信息

数据描述：商家编号、密码、名称、地址

存取方式：按照商家编号随机存取

（3）菜品：存储所有商家的菜品信息

数据描述：菜品编号、商家编号、菜品名称、价格

存取方式：按照商家编号和菜品编号进行查询

（4）订单信息

含义说明：用于存储订单的整体信息及订单状态，是系统业务流转的核心数据存储。

数据描述：订单编号、商家编号、客户编号、外卖员编号、下单时间、订单金额、订单状态

存取方式：根据订单编号和商家编号进行查询